

Despensa Inteligente

Gestión agéntica de inventario doméstico y prevención de desperdicio de alimentos

Inteligencia Artificial Aplicada a Organizaciones — UTN FRBA Trabajo Práctico Final · Entrega: 16/08/2026 Autor: Pablo Monsalvo

Links del proyecto

Recurso	URL
Aplicación en vivo (HTTPS, abrir desde el celular)	https://pablomonsalvo76.github.io/despensa-inteligente/
Repositorio público (código e historia de commits)	https://github.com/pablomonsalvo76/despensa-inteligente
Documentación técnica completa	https://github.com/pablomonsalvo76/despensa-inteligente/tree/main/docs

La cámara requiere HTTPS: los navegadores bloquean el acceso a `getUserMedia` sobre HTTP simple. El link de arriba ya lo cumple. La app se puede instalar con "Agregar a pantalla de inicio" y queda como una aplicación nativa más.

Anexos (documentos completos en el repositorio, referenciados a lo largo del informe):

Anexo	Contenido
<code>docs/DIAGRAMAS.md</code>	Arquitectura general, flujo de agentes y tres diagramas UML
<code>docs/UX_NIELSEN.md</code>	Evaluación de las 10 heurísticas de Nielsen
<code>docs/CIBERSEGURIDAD.md</code>	7 riesgos con evidencia y mitigación
<code>docs/PARTE_2_IA_LOCAL.md</code>	Parte 2 — IA local, las 4 preguntas
<code>docs/EQUIPO_Y_COWORK.md</code>	Secciones 1 y 7 — equipo e IA en el desarrollo
<code>docs/CONTEXTO.md</code>	Registro vivo de decisiones, errores y correcciones

1. El equipo

Trabajo individual. El detalle de cómo se organizó el desarrollo y qué papel cumplió cada herramienta de IA está en el Anexo `EQUIPO_Y_COWORK.md` y se resume en la Sección 7 de este informe.

2. El problema

En un hogar se tira comida por dos razones que no tienen que ver con la falta de ganas de cocinar: **no se sabe qué hay**, y **no se sabe qué está por vencer**. Cuando uno se entera, ya es tarde.

La app ataca eso con un ciclo cerrado —**compra** → **despensa** → **consumo** → **compra**— sostenido por agentes que se reparten las decisiones: registra lo que entra, vigila los vencimientos, propone qué cocinar priorizando lo que está por vencerse, aprende de la conducta real del usuario, y le dice qué comprar y qué **no** comprar, porque comprar de más es una de las causas del problema, no una solución.

Público objetivo: usuarios domésticos que gestionan la despensa de su hogar, sin conocimiento técnico particular. Como escalamiento documentado (Sección 8), personal de cocina de un hotel o restaurante.

3. Arquitectura

Doce agentes, cada uno en su archivo, coordinados por un orquestador que corre el ciclo **Observación** → **Análisis** → **Planificación** → **Acción** → **Evaluación** → **Aprendizaje**. Ese ciclo no es una figura del informe: es visible en la app, en la pestaña *Sistema*, con el log real de cada corrida.

Agente	Qué decide
Inventario	Alta, baja y estado de cada producto
Vencimientos	Semáforo de urgencia y umbrales de alerta
Captura	Escaneo de código de barras, OCR de fecha, carga manual
Conversacional	Interpreta lenguaje natural del chat
Hogar	Perfil de los comensales: alergias y condiciones médicas
Cocinero	Qué recetas proponer y en qué orden
Generador	Inventa recetas nuevas con un LLM, bajo veto determinístico
Compras	Qué comprar y qué no comprar
Evaluador	Registra el desenlace de cada decisión
Aprendizaje	Ajusta umbrales, gustos y estilo desde la conducta
Impacto	KPIs: rescatado vs. desperdiciado, ahorro estimado
Orquestador	Corre el ciclo y comunica a los demás

Los diagramas (arquitectura general, flujo de agentes, y los UML de secuencia, casos de uso y modelo de datos) están en el Anexo `DIAGRAMAS.md`.

Tabla de stack

Capa	Tecnología	Por qué
Interfaz	HTML + CSS + JavaScript sin framework ni build	Una PWA de este tamaño no necesita build; se despliega como archivos estáticos y arranca instantáneo
Distribución	PWA sobre GitHub Pages	HTTPS gratis (requisito de la cámara), instalable, sin servidor que mantener
Persistencia	<code>localStorage</code> vía <code>js/db.js</code> , con exportar/importar	Ver Sección 6: los datos de salud del hogar nunca salen del dispositivo
Código de barras	<code>html5-qrcode</code> (EAN-13/EAN-8/UPC)	Resuelve el GTIN contra Open Food Facts, API pública sin credenciales
OCR (principal)	PP-OCR (PaddleOCR) sobre ONNX Runtime Web	100% en el dispositivo, sin enviar la foto a ningún servidor
OCR (respaldo)	Tesseract.js v5	Último recurso; 0 aciertos medidos en fechas troqueladas reales
IA generativa	Gemini vía Firebase AI Logic; Ollama como alternativa local	Ver Parte 2
Pruebas	Node sin dependencias externas	335 pruebas en 8 suites, ejecutables con <code>node tests/*.test.js</code>

La decisión arquitectónica central: dos capas

El sistema de recetas tiene **un piso y un techo**, y esa separación explica casi todas las demás decisiones:

- **Recetario local** = piso garantizado. Responde siempre, sin conexión, sin modelo y sin costo. Nunca se apaga.
- **Generación con LLM** = techo. Cuando hay un modelo disponible, propone recetas que el recetario no podría cubrir.

Esa separación dejó de ser teórica durante el desarrollo. El 15/08 se agotó la cuota gratuita del modelo (HTTP 429) en pleno uso, y el recetario local siguió respondiendo sin interrupción. El incidente está documentado en `CONTEXTO.md` con el error textual.

El principio que ordena el código: el modelo propone, el código veta

Ninguna receta generada por el LLM llega al usuario sin pasar por `validar()`, una función pura, sin red, testeada aparte con salidas adversarias:

- **Lista cerrada:** sólo puede usar ingredientes que estén realmente en la despensa. Un ingrediente inventado se rechaza y con él la receta entera.
- **Los tags que el modelo declara sobre sí mismo se descartan** y se recalculan desde los ingredientes. Si el modelo afirma que una receta con pollo es vegana, no se le cree.
- **Las alergias son filtro duro**, no puntaje.

- Un producto vencido nunca entra en el pool de ingredientes, así que el modelo no puede proponerlo aunque quiera.

Esto no es una precaución genérica: la app guarda **alergias y condiciones médicas** de los comensales. Un modelo que alucina un ingrediente es un riesgo real, no una molestia.

4. Evidencia de funcionamiento

Doce capturas tomadas el 16/08/2026 desde un teléfono Android real, sobre la app publicada en HTTPS y con datos cargados por el usuario. El índice completo está en `docs/capturas/README.md`. Las que sostienen la evaluación:

Captura	Qué demuestra
<code>captura 2</code>	Inicio con estado real: productos por vencer, recetas disponibles, resumen por categoría
<code>captura 5</code> + <code>captura 6</code>	El flujo principal, y la arquitectura en dos capas en dos imágenes: el OCR local no logra leer una fecha impresa sobre plástico arrugado, lo dice sin inventar nada y ofrece el respaldo de visión detrás de un botón explícito; la siguiente muestra los tres pasos resueltos
<code>captura 9</code>	Priorización por urgencia: <i>"Rescata lo más urgente: Milanese"</i>
<code>captura 10</code>	Receta generada con la etiqueta <code>inventada</code> , pasos completos y el aviso de que pasó los mismos controles que las del recetario
<code>captura 11</code>	<i>"Se hace con milanese que se te vence y papa frita que tenés en la alacena. Te falta..."</i> — el sistema dice qué hay en casa, dónde , y qué falta comprar
<code>captura 12</code>	Sistema: ciclo de orquestación y log en vivo de los agentes

Log de sesión real: `docs/capturas/despensa-inteligente-2026-08-16.json`, exportación completa de la memoria del sistema desde *Perfil* → *Copia de seguridad*. No es una captura: es el estado real de todos los almacenes.

Dos detalles que un evaluador atento puede verificar en las imágenes:

- En `captura 10`, entre los ingredientes de la receta generada aparece *"mayonesa hellmanns clasica"*. Ese producto expuso un defecto real del validador el 15/08 —rechazaba la respuesta correcta del modelo por una diferencia de apóstrofo entre el prompt saneado y la lista cerrada— documentado en `CONTEXTO.md`. La captura es la verificación en producción de que el arreglo funciona.
- En `captura 9`, *"Arroz guisado con asado desmenuzado"* **no** es una de las 27 recetas escritas a mano: es una receta generada por el modelo, validada e incorporada al catálogo, conviviendo con las fijas.

Qué es real y qué no

Una app de demostración que finge funcionalidades no sirve para evaluar nada. Esta tabla dice exactamente qué está implementado:

Función	Estado
Inventario, semáforo, alertas, historial, log del ciclo	Real, con persistencia local
Escaneo de código de barras	Real: cámara + resolución de GTIN contra Open Food Facts
OCR de fecha de vencimiento	Real: PP-OCR en el navegador, sin enviar la imagen a ningún servidor
Recetas del recetario	Real, y el recetario crece (ver abajo)
Generación de recetas nuevas	Real, con LLM y veto determinístico
Respaldo de visión para fechas difíciles	Real, detrás de un botón explícito. Nunca automático
Chatbot	Real, parser por reglas. No entiende lenguaje libre arbitrario
Aprendizaje de gustos y estilo	Real y medible (Sección 5)
Notificaciones	Del navegador. No son push de servidor

El recetario dejó de ser fijo

El recetario arranca con 27 recetas curadas paso a paso, que en conjunto usan 35 ingredientes. Ese techo era real: cargar quinoa, palta y kiwi devolvía **cero** recetas.

Cada receta que el modelo genera y que el validador aprueba se incorpora ahora al catálogo. Desde ese momento aparece en el ranking junto a las originales, queda disponible **offline y sin consumir cuota**, y le enseña sus ingredientes al motor de equivalencias — de modo que el techo de los 35 sube con el uso en vez de quedar clavado en lo que alguien escribió a mano.

Verificación

335 pruebas en 8 suites, sin dependencias externas. Se verificaron **por mutación**: se rompió cada control a propósito en una copia para confirmar que la suite lo detecta. Un test que pasa igual con el código roto no es evidencia de nada. Ese proceso encontró dos pruebas que no discriminaban y se reescribieron.

5. UX/UI — Heurísticas de Nielsen

Se evaluaron las **10 heurísticas**, no el mínimo de 5 pedido, porque la app tiene evidencia concreta para cada una. Resultado: **7 completas, 3 parciales, ninguna incumplida**. El desarrollo completo está en el Anexo `UX_NIELSEN.md`.

Las tres parciales comparten causa: son las partes más nuevas del desarrollo, construidas bajo presión de tiempo y sin una segunda pasada de pulido.

Un caso concreto: cuando la transparencia destruyó la confianza

Durante el desarrollo, el panel del escáner mostraba el texto en bruto que interpretaba el reconocedor, rotulado «**La cámara lee: ...**». La intención era transparencia. El efecto fue el contrario.

El rótulo prometía algo falso —la cámara no lee, el OCR interpreta— y lo que mostraba era el intento del reconocedor *mientras todavía estaba fallando* ("0" , "-."). El usuario lo comparaba con lo que veía en pantalla, nunca coincidía, y concluía que la app estaba rota.

Se corrigió con divulgación progresiva: el dato se conserva, colapsado, bajo el rótulo honesto "*ver el texto en bruto que interpretó el lector*". Es un ejemplo de que una heurística no se cumple por agregar información, sino por agregar la información correcta con la promesa correcta.

Prueba con usuario real

Prueba informal el **16/08/2026** con una persona que no había visto la app antes, en su propio teléfono, sin indicaciones ni acompañamiento. Detalle completo en el Anexo `UX_NIELSEN.md`, sección 5.2.

Lo que se verificó sin intervención del evaluador: entendió el semáforo de colores sin explicación, identificó las tres vías de carga y las leyó como flexibilidad —no como complejidad—, y comprendió que las recetas se arman con el inventario real, destacando que se marquen los ingredientes faltantes.

Lo que propuso: marcar recetas como favoritas y derivar de ellas la lista de compras.

Por qué la propuesta importa. Revela un hueco real: hoy el sistema tiene una señal negativa explícita —descartar una receta— pero la única señal positiva explícita es cocinarla, lo que exige efectivamente cocinar. Un favorito permitiría decir "esta me gusta" antes de tener los ingredientes, que es justo cuando el dato sirve para decidir la compra.

Y trae una tensión que conviene nombrar. La tesis de la app es cocinar lo que está por vencer y **comprar menos**; una lista derivada de recetas deseadas empuja a comprar más. Hoy el Agente de Compras evita eso con dos restricciones: sólo propone comprar para recetas a las que faltan como máximo dos ingredientes, y sólo si la receta rescata algo en riesgo. La resolución propuesta es implementar favoritos como **señal de aprendizaje** y mantener la lista de compras subordinada al rescate: los favoritos ordenan las sugerencias, pero no habilitan comprar para una receta que no rescata nada.

Alcance honesto: una sesión con una persona y sin obstáculos encontrados es evidencia limitada. No prueba que la app sea fácil de usar; prueba que este usuario, en esta sesión, no se trabó. Una segunda ronda debería buscar lo que ésta no cubrió: el comportamiento tras dos fallos seguidos del OCR, si se entiende la diferencia entre *eliminar* y *descartar* —conceptualmente sutil y con efecto sobre las métricas—,

y si el usuario encuentra "Leer con IA" sin que se la señalen, que es la falla ya identificada en la heurística 8.

Cuánto aprende el sistema, medido

"El agente aprende" es una afirmación vacía si no se puede medir. Estos son números reales del sistema:

Medición	Valor
Dimensiones que aprende	3 (cocina, estilo, tipo) sobre 14 valores posibles
Perfil tras cocinar 3 recetas italianas	<pre>{"cocina":{"italiana":3},"estilo":{"casera":2,"elaborada":1},"tipo":{"principal":3}}</pre>
Peso máximo del gusto aprendido en el ranking	6 puntos
Peso de un producto que vence mañana	~20 puntos
Rechazos necesarios para penalizar un ingrediente	3 recetas distintas

El tope de 6 contra 20 es la decisión de diseño más importante del motor de recomendación. El gusto puede ordenar entre recetas que ya son viables, pero nunca puede tapar una que rescata un producto a punto de vencer. La app existe para que no se tire comida, no para adivinar el antojo del usuario.

Por el mismo criterio, el último lugar de cada lista se reserva para una receta que **no** se está beneficiando del gusto aprendido: un recomendador que sólo refuerza lo que ya sabe encierra al usuario y deja de aprender.

6. Ciberseguridad

Siete riesgos identificados con evidencia y mitigación, sobre el mínimo de 4 pedido. El desarrollo completo está en el Anexo `CIBERSEGURIDAD.md`. Los tres que definen la arquitectura:

Datos de salud. La app guarda alergias y condiciones médicas de los comensales del hogar. Regla general: **nunca salen del dispositivo**, porque no hay backend que los almacene. Excepción documentada y con consentimiento explícito: cuando el usuario activa la generación de recetas con IA en la nube, las alergias viajan en el prompt para que el modelo las respete.

Secretos en el cliente. La app no tiene backend, así que cualquier configuración llega necesariamente al navegador. Se manejó en capas: la clave de Firebase no es secreta por diseño, pero se codificó para no dejarla grepeable por bots que escanean GitHub; y la clave real está **restringida por dominio** en Google Cloud Console, así que copiarla no sirve fuera de los orígenes reales de la app.

Una degradación honesta. La verificación de aplicación (Firebase App Check) se pensó sobre reCAPTCHA v3, que resultó tener un fallo reproducible del propio SDK de Firebase. Se documentó el trade-off en vez de esconderlo: se usa un token de depuración, que es una protección más débil, con la restricción por dominio como respaldo. La alternativa era que la funcionalidad no funcionara en absoluto.

7. IA en el desarrollo (co-work) y reflexión

Detalle completo en el Anexo `EQUIPO_Y_COWORK.md`. La conclusión que vale la pena adelantar acá:

Las IAs se equivocaron, y documentarlo es parte del trabajo. Durante la integración del modelo en la nube, tres asistentes distintos propusieron diagnósticos incorrectos sobre un mismo error, y el problema real resultó ser otro. La lección no es que la IA sirva o no sirva: es que **el humano tiene que saber qué preguntar y por qué la respuesta puede estar mal**. Sin la capacidad de verificar contra el error observado, se habrían aplicado tres "soluciones" que no arreglaban nada.

Ese criterio se llevó al código: la regla *el modelo propone, el código veta* es exactamente la misma idea aplicada al producto.

8. Escalamiento a gastronomía (hotel / restaurante)

El caso hogareño fue el punto de partida para **validar el ciclo completo** con datos reales y volumen manejable. El mismo motor sirve para una cocina profesional, pero hay cinco cosas que cambian y conviene nombrarlas con precisión, porque no son ajustes de interfaz:

1. Unidades y magnitudes. El hogar razona en "1 paquete", "2 unidades". Una cocina razona en kilos, litros y porciones por servicio. El modelo de datos ya tiene `quantity`, pero necesitaría unidad y conversión, y las recetas necesitarían rendimiento por comensal.

2. Costo real. En el hogar el desperdicio se mide en comida tirada. En un restaurante se mide en dinero y en margen por plato. El Agente de Impacto ya calcula ahorro estimado; ahí pasaría a ser un indicador de gestión, con precio de compra por lote.

3. Roles y multiusuario. Hoy cada instalación es de una persona, sin cuentas. Una cocina tiene jefe de cocina, encargado de compras y personal de salón, con permisos distintos. Esto es lo que obliga a cambiar la persistencia (ver Sección 9): datos compartidos, autenticación y control de acceso.

4. PEPS y trazabilidad de lote. Primero entrado, primero salido, es normativa, no una sugerencia. El sistema ya prioriza por vencimiento —que es la misma idea— pero necesitaría número de lote y proveedor por partida para poder responder ante un retiro de producto del mercado.

5. Alérgenos de carta. El Agente de Hogar ya modela alergias de los comensales y bloquea recetas de forma dura. En gastronomía eso se invierte: no se conoce al comensal, así que hay que **declarar** los alérgenos de cada plato. Es el mismo dato, expuesto en la dirección contraria.

De los cinco, el único que exige rehacer una decisión estructural es el 3. Los otros cuatro son extensiones del modelo de datos actual.

9. Limitaciones conocidas y trabajo futuro

Un informe que no nombra sus límites no es honesto. Estos están identificados, documentados en el repositorio y **no** improvisados para esta sección:

El desenlace 'ignorado' no está implementado. El Evaluador reconoce hoy dos desenlaces —cocinado y descartado— y falta un tercero: el producto que quedó sin usarse, que no se cocinó ni se tiró. Sin esa señal, el sistema no puede decir *"este producto no lo consumís tanto, no deberías tener tanto stock"*. Es la funcionalidad pendiente de mayor valor y su diseño ya está planteado.

El recetario fijo no distingue cortes. Tiene un único ingrediente genérico `carne`, así que un asado y un peceto son intercambiables para él, y puede proponer milanesas con un corte de parrilla. La capa de equivalencias resuelve la **disponibilidad** ("¿tengo carne?") pero pierde el detalle. Notablemente, la capa generativa **no** tiene este problema: al modelo se le entrega el nombre real del producto, así que sabe qué es un asado. Es la demostración concreta de por qué existen las dos capas.

No hay señal positiva explícita sobre una receta. Descartar una receta es una acción disponible; marcarla como deseada, no. La única forma de decirle al sistema que algo gusta es cocinarlo. Es el hueco que detectó la prueba con usuario (Sección 5) y la mejora de mayor valor por esfuerzo que queda pendiente.

El inventario cuenta unidades, no peso. Cada producto tiene una cantidad entera, y cocinar una receta descuenta una unidad. Para un paquete de fideos o un sachet de leche eso es correcto; para la carne no, porque se compra y se consume por kilo. La consecuencia práctica es que medio kilo de carne usado en una receta descuenta el producto entero. Es la misma carencia que la Sección 8 nombra como primer cambio necesario para gastronomía —unidad y conversión en el modelo de datos— y aparece también en el caso doméstico.

Persistencia local. `localStorage` vive en un dispositivo: no se comparte entre teléfonos y se pierde si se borran los datos del navegador. Está mitigado con exportar/importar. Fue una decisión consciente por privacidad de datos de salud y por funcionamiento offline, no una omisión. La evolución natural es IndexedDB primero —sigue siendo local, pero asíncrono y con índices— y una base compartida sólo si se toma el camino de la Sección 8.

Cuota del modelo. La IA en la nube corre sobre un plan gratuito con tope diario, compartido entre la generación de recetas y el lector de fechas. Al agotarse, la app sigue funcionando completa sobre su capa local.

10. Conclusión

El objetivo no era construir una app que pareciera inteligente, sino una que **tome decisiones verificables** sobre un problema real y acotado. Las tres cosas que sostienen esa afirmación:

1. **El ciclo de agentes es real y observable**, no una figura del informe: se puede abrir la pestaña Sistema y ver el log de cada corrida.
2. **La inteligencia está acotada a propósito**. El gusto aprendido pesa 6 contra 20 de la urgencia; el modelo generativo no puede nombrar un ingrediente que no esté en la despensa; una receta vencida no se ofrece nunca. Un sistema que puede hacer cualquier cosa no se puede evaluar.
3. **Los errores están documentados, no escondidos**. El registro de desarrollo incluye los diagnósticos equivocados, los cambios que se revirtieron y las limitaciones que quedaron abiertas.

La app funciona hoy, en un teléfono real, sobre HTTPS, con datos reales. El link está en la primera página.